SQL JOIN OPERATIONS

DBMS Study Notes | Definitions, Syntax, Examples & Exam Questions

Learning objective: Understand how SQL combines rows from two or more tables using related columns.

Beginner-friendly notes for postgraduate DBMS preparation

1. What is a JOIN?

A **JOIN** is an SQL operation used to retrieve related data from two or more tables. It combines rows based on a matching condition, usually a primary key in one table and a foreign key in another.

Example: A college database may store student details in a STUDENT table and course details in a COURSE table. A JOIN can display a student's name together with the course information.

Sample tables used in this note

STUDENT	student_id	student_name	course_id
	1	Anu	C1
	2	Binu	C2
	3	Ciya	C3
	4	Deepa	NULL

COURSE	course_id	course_name
	C1	Computer Science
	C2	Mathematics
	C4	Physics

The common column is **course_id**. It is used to match students with courses.

General JOIN syntax

```
SELECT column1, column2
FROM table1
JOIN table2
ON table1.common_column = table2.common_column;
```

The **ON** clause specifies the condition used to match rows. Always qualify a column with its table name when both tables contain a column with the same name.

2. INNER JOIN

An **INNER JOIN** returns only rows that have matching values in both tables. Rows without a match are excluded.

```
SELECT s.student_name, c.course_name
FROM STUDENT s
INNER JOIN COURSE c
ON s.course_id = c.course_id;
```

Result

student_name	course_name
Anu	Computer Science
Binu	Mathematics

Ciya has course_id C3, which does not exist in COURSE. Deepa has no course_id. The course C4 has no matching student. These rows do not appear in the INNER JOIN result.

Remember: INNER JOIN = matching rows only.

3. LEFT OUTER JOIN (LEFT JOIN)

A **LEFT JOIN** returns every row from the left table (the table written before LEFT JOIN) and matching rows from the right table. If there is no match, the right-table columns contain NULL.

```
SELECT s.student_name, c.course_name
FROM STUDENT s
LEFT JOIN COURSE c
ON s.course_id = c.course_id;
```

Result

student_name	course_name
Anu	Computer Science
Binu	Mathematics
Ciya	NULL
Deepa	NULL

Remember: LEFT JOIN = all left-table rows + matching right-table data.

4. RIGHT OUTER JOIN (RIGHT JOIN)

A **RIGHT JOIN** returns every row from the right table and matching rows from the left table. If there is no match, the left-table columns contain NULL.

```
SELECT s.student_name, c.course_name
FROM STUDENT s
RIGHT JOIN COURSE c
ON s.course_id = c.course_id;
```

Result

student_name	course_name
Anu	Computer Science
Binu	Mathematics
NULL	Physics

Remember: RIGHT JOIN = all right-table rows + matching left-table data.

5. FULL OUTER JOIN

A **FULL OUTER JOIN** returns all rows from both tables. Matching rows are combined. Non-matching rows from either table are also included, with NULL in the columns from the table that has no match.

```
SELECT s.student_name, c.course_name
FROM STUDENT s
FULL OUTER JOIN COURSE c
ON s.course_id = c.course_id;
```

Conceptual result

student_name	course_name
Anu	Computer Science
Binu	Mathematics
Ciya	NULL

student_name	course_name
Deepa	NULL
NULL	Physics

Important: MySQL does not directly support the FULL OUTER JOIN syntax. It can be emulated using a LEFT JOIN combined with a RIGHT JOIN using UNION (or an equivalent UNION ALL pattern with duplicate handling).

```
SELECT s.student_name, c.course_name
FROM STUDENT s
LEFT JOIN COURSE c ON s.course_id = c.course_id
UNION
SELECT s.student_name, c.course_name
FROM STUDENT s
RIGHT JOIN COURSE c ON s.course_id = c.course_id;
```

UNION removes duplicate rows. Use UNION ALL only when you intentionally want to keep duplicates and have handled matched rows appropriately.

6. CROSS JOIN

A **CROSS JOIN** returns the Cartesian product: every row of the first table is combined with every row of the second table. It does not require a matching condition.

```
SELECT s.student_name, c.course_name
FROM STUDENT s
CROSS JOIN COURSE c;
```

If STUDENT contains 4 rows and COURSE contains 3 rows, the result contains **4 × 3 = 12 rows**.

Use CROSS JOIN carefully because the result can become very large.

7. SELF JOIN

A **SELF JOIN** joins a table with itself. It is useful when rows in the same table are related, such as employees and their managers. Table aliases are used to distinguish the two copies.

Example table: EMPLOYEE(employee_id, employee_name, manager_id). The manager_id refers to another employee_id in the same table.

```
SELECT e.employee_name AS employee,
       m.employee_name AS manager
FROM EMPLOYEE e
LEFT JOIN EMPLOYEE m
ON e.manager_id = m.employee_id;
```

The LEFT JOIN also displays employees who do not have a manager; their manager value appears as NULL.

8. JOIN types at a glance

JOIN type	Rows returned	Unmatched rows included?
INNER JOIN	Only matching rows	No
LEFT JOIN	All left + matching right	Unmatched left rows
RIGHT JOIN	All right + matching left	Unmatched right rows
FULL OUTER JOIN	All rows from both sides	Unmatched rows from both
CROSS JOIN	Every possible row combination	All combinations

JOIN type	Rows returned	Unmatched rows included?
SELF JOIN	Rows matched within same table	Depends on join type used

Memory trick

LEFT = keep everything on the left. RIGHT = keep everything on the right. FULL = keep everything on both sides. INNER = keep only the overlap.

9. Important SQL points

• **ON:** Specifies the condition used to match rows.
• **Alias:** A temporary short name for a table, such as STUDENT s.
• **NULL:** Represents missing or unknown data; it is not the same as zero or an empty string.
• **UNION:** Combines the results of two compatible SELECT statements and removes duplicates.
• **UNION ALL:** Combines results while retaining duplicates.
• **Foreign key:** A column that refers to a key in another table and often provides the relationship used in a JOIN.

10. Exam questions with answers

1. Define SQL JOIN.
A JOIN is an SQL operation used to combine rows from two or more tables based on a related column or matching condition.

2. What is the difference between INNER JOIN and LEFT JOIN?
INNER JOIN returns only matching rows from both tables. LEFT JOIN returns all rows from the left table and matching rows from the right table; unmatched right-side values are shown as NULL.

3. Which JOIN returns all records from both tables?
FULL OUTER JOIN.

4. What is a Cartesian product?
It is the result of combining every row of one table with every row of another table. CROSS JOIN produces this result.

5. What is a SELF JOIN?
A SELF JOIN joins a table with itself, usually using aliases to compare related rows.

6. Does MySQL support FULL OUTER JOIN directly?
No. It can be simulated using LEFT JOIN and RIGHT JOIN combined with UNION.

7. What is the purpose of the ON clause?
It specifies the condition that determines which rows from the tables should match.

8. If table A has 5 rows and table B has 4 rows, how many rows can CROSS JOIN return?
5 × 4 = 20 rows.

11. Practice questions

Try these without looking at the answers:

1. Which JOIN returns only matching records?
2. Which JOIN keeps all rows from the left table?
3. Which JOIN produces a Cartesian product?
4. Which JOIN is used to compare rows within the same table?
5. What can be used in MySQL to simulate FULL OUTER JOIN?

Answer key: 1. INNER JOIN; 2. LEFT JOIN; 3. CROSS JOIN; 4. SELF JOIN; 5. LEFT JOIN + RIGHT JOIN with UNION.

12. Final revision

Keyword	Quick meaning
INNER	Only matches
LEFT	All left rows
RIGHT	All right rows
FULL	All rows from both tables
CROSS	All combinations
SELF	Table joined to itself

Study tip: Practise writing each JOIN query and predicting its output using small sample tables. In exams, clearly identify which table is on the left and which is on the right.